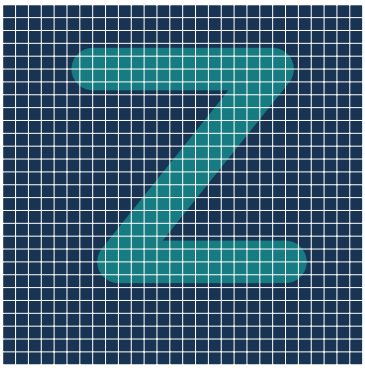
MATHEMATICS OF HANDWRITTEN DIGIT RECOGNITION

A complete lecture course using MNIST (28×28)



Linear Algebra • Calculus • Probability • Optimization • CNNs

Lecture Notes and Worked Problems

Includes full derivations, dimension checks, numerical calculations, implementations, and exercises

October 2026

Preface

These notes explain handwritten digit recognition from first mathematical principles. The goal is to understand exactly what is multiplied, differentiated, optimized, and evaluated, rather than merely calling a deep-learning library. The treatment starts with pixels and a linear classifier, derives multilayer backpropagation, then develops convolutional neural networks and practical training. Small worked calculations are deliberately included so every step can be checked by hand. Familiarity with matrix multiplication and elementary derivatives is helpful but the relevant tools are reviewed.

Contents

Preface **i**

1 The Problem and Its Mathematical Language **1**

1.1 What is MNIST? . . . 1

1.2 The four mathematical ingredients . . . 1

1.3 Index conventions and dimensions . . . 1

1.4 Pixel normalization and vectorization . . . 1

2 Linear Classification and Probability **3**

2.1 Scores from weighted pixels . . . 3

2.2 Softmax: ten scores become probabilities . . . 3

2.3 One-hot target labels . . . 3

2.4 Cross-entropy and maximum likelihood . . . 3

2.5 Stable computation using log-sum-exp . . . 4

3 Multilayer Perceptrons: Full Forward Mathematics **5**

3.1 Why hidden layers? . . . 5

3.2 Network architecture: 784–128–64–10 . . . 5

3.3 ReLU and its derivative . . . 5

3.4 Counting trainable parameters . . . 5

3.5 What neurons actually calculate . . . 6

4 Calculus and Backpropagation **7**

4.1 Gradient descent . . . 7

4.2 Deriving the softmax Jacobian . . . 7

4.3 Why softmax plus cross-entropy simplifies . . . 7

4.4 Output-layer gradients . . . 8

4.5 Hidden-layer gradients . . . 8

4.6 Small numerical backpropagation example . . . 8

4.7 Gradient checking with finite differences . . . 8

5 Convolutional Neural Networks **9**

5.1 Local patterns and weight sharing 9
5.2 A convolution-like operation: cross-correlation 9
5.3 Output-size mathematics 9
5.4 Multiple input channels and output maps 10
5.5 Activation and pooling 10
5.6 Full MNIST CNN: dimensions and parameter count 10
5.7 Backpropagation through convolution 11
5.8 Gradient through max pooling 11

6 Training Algorithms and Generalization **12**
6.1 Mini-batch optimization 12
6.2 Batch matrix gradient formulas 12
6.3 Adam optimizer 12
6.4 Initialization and regularization 12
6.5 Train, validation and test sets 13

7 Evaluation, Inference and Implementations **14**
7.1 Accuracy and confusion matrix 14
7.2 New-image inference pipeline 14
7.3 Clear PyTorch implementation 14

8 Review Exercises with Fully Worked Answers **16**

9 Final Formula Sheet **18**

Chapter 1

The Problem and Its Mathematical Language

1.1 What is MNIST?

MNIST is a supervised dataset of 60,000 training and 10,000 test grayscale handwritten digit images, labeled from 0 through 9. Each image is 28×28, with grayscale pixel values in $\{0, \ldots, 255\}$. Typical images show light ink on a dark background. Each image contains $28(28) = 784$ scalar brightness values.

$$X = \begin{bmatrix} x_{1,1} & \cdots & x_{1,28} \\ \vdots & \ddots & \vdots \\ x_{28,1} & \cdots & x_{28,28} \end{bmatrix} \in \mathbb{R}^{28\times 28}, \qquad y \in \{0, 1, \ldots, 9\}. \tag{1.1}$$

The learning problem is to find a function f_θ with trainable parameters θ that maps X to class scores and eventually ten probabilities. It is not enough to perform well on training images: we seek correct predictions on unseen handwriting.

1.2 The four mathematical ingredients

1. **Linear algebra**: vectors, matrices, and tensors encode pixel data and layer weights.
2. **Calculus**: derivatives tell us how each weight affects the error.
3. **Probability**: a classifier predicts a distribution over ten mutually exclusive classes.
4. **Optimization**: an algorithm modifies weights to reduce measured error.

1.3 Index conventions and dimensions

We use bold lower-case for column vectors, capitals for matrices, and superscript (ℓ) for a network layer. Input features are indexed $j = 1, \ldots, 784$; output classes use $k = 0, \ldots, 9$. A common source of mistakes is multiplying shapes in the wrong order. If $W \in \mathbb{R}^{m\times n}$ and $\boldsymbol{x} \in \mathbb{R}^n$, then $W\boldsymbol{x} \in \mathbb{R}^m$.

1.4 Pixel normalization and vectorization

First convert a pixel $x_{i,j}$ to $x'_{i,j} = x_{i,j}/255 \in [0, 1]$. A pixel value of 204 becomes $204/255 = 0.8$. For a multilayer perceptron (MLP), flatten in row-major order: $j = 28(i-1) + c$ for image row

i and column c. The normalized vector $\boldsymbol{x} \in \mathbb{R}^{784}$ retains values but does not explicitly preserve two-dimensional neighborhood structure. A CNN instead accepts $X \in \mathbb{R}^{1\times 28\times 28}$.

Worked Example

For a miniature 2×3 image, $X = \begin{bmatrix} 0 & 255 & 128 \\ 64 & 255 & 0 \end{bmatrix}$, row-major flattening and normalization give $\boldsymbol{x} = (0, 1, 128/255, 64/255, 1, 0)^T$. The same procedure applies to the 784 entries of MNIST.

Chapter 2

Linear Classification and Probability

2.1 Scores from weighted pixels

The simplest classifier assigns a separate weight to each pixel for each digit:

$$\boldsymbol{z} = W\boldsymbol{x} + \boldsymbol{b}, \quad W \in \mathbb{R}^{10\times 784}, \quad \boldsymbol{b} \in \mathbb{R}^{10}, \quad z_k = \sum_{j=1}^{784} w_{kj}x_j + b_k. \tag{2.1}$$

There are $10(784) + 10 = 7{,}850$ parameters. A score (logit) can be negative, positive, or zero and is not itself a probability.

2.2 Softmax: ten scores become probabilities

$$\boxed{p_k = \frac{\exp(z_k)}{\sum_{j=0}^{9}\exp(z_j)}} \qquad \sum_{k=0}^{9} p_k = 1, \quad 0 < p_k < 1. \tag{2.2}$$

The predicted digit is $\hat{y} = \arg\max_k p_k = \arg\max_k z_k$: exponentiation and normalization preserve score ordering.

Worked Example

If three illustrative logits are $\boldsymbol{z} = (1, 2, 0)$, then $e^1 = 2.7183$, $e^2 = 7.3891$, $e^0 = 1$. The sum is 11.1074 and $\boldsymbol{p} \approx (0.2447, 0.6652, 0.0900)$. The predicted class is the middle class. Actual MNIST softmax has ten entries, not three.

2.3 One-hot target labels

For a true digit 5, use $\boldsymbol{y} = (0, 0, 0, 0, 0, 1, 0, 0, 0, 0)^T$. In general $y_k = 1$ only when $k = t$, the correct class. This is a categorical target, not a regression target equal to the numeric digit.

2.4 Cross-entropy and maximum likelihood

$$\boxed{L = -\sum_{k=0}^{9} y_k \log p_k = -\log p_t.} \tag{2.3}$$

A correct-class probability 0.9 gives $L = 0.10536$, while 0.01 gives $L = 4.60517$. The negative log penalizes confidently incorrect predictions heavily. For N independent examples, the conditional likelihood is $\mathcal{L}(\theta) = \prod_{i=1}^{N} p_{y_i}(X_i; \theta)$. Taking $-\log$ and dividing by N yields

$$J(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log p_{y_i}(X_i; \theta), \tag{2.4}$$

so minimizing cross-entropy is equivalent to maximizing conditional log-likelihood.

Important Note

Softmax outputs are model probabilities but need not be calibrated. A model can report 99% confidence and still be wrong. Evaluate both accuracy and calibration when confidence matters.

2.5 Stable computation using log-sum-exp

Large logits can make e^z overflow. Put $m = \max_j z_j$:

$$p_k = \frac{e^{z_k - m}}{\sum_j e^{z_j - m}}, \quad L = -z_t + m + \log \left(\sum_j e^{z_j - m} \right). \tag{2.5}$$

Always pass raw logits to framework cross-entropy implementations; they typically use a stable formula internally.

Chapter 3

Multilayer Perceptrons: Full Forward Mathematics

3.1 Why hidden layers?

A single affine map can only build linear decision boundaries between classes. Real handwritten strokes and loops have nonlinear relationships; activation functions allow the model to compose nonlinear features.

3.2 Network architecture: 784–128–64–10

$$\boldsymbol{z}^{(1)} = W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}, \qquad \boldsymbol{a}^{(1)} = \mathrm{ReLU}(\boldsymbol{z}^{(1)}), \tag{3.1}$$

$$\boldsymbol{z}^{(2)} = W^{(2)}\boldsymbol{a}^{(1)} + \boldsymbol{b}^{(2)}, \qquad \boldsymbol{a}^{(2)} = \mathrm{ReLU}(\boldsymbol{z}^{(2)}), \tag{3.2}$$

$$\boldsymbol{z}^{(3)} = W^{(3)}\boldsymbol{a}^{(2)} + \boldsymbol{b}^{(3)}, \qquad \boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z}^{(3)}). \tag{3.3}$$

The shapes are $W^{(1)} : 128 \times 784$, $W^{(2)} : 64 \times 128$, and $W^{(3)} : 10 \times 64$.

Key Result

$$\boldsymbol{p} = \mathrm{softmax}\Big(W^{(3)}\,\mathrm{ReLU}\Big(W^{(2)}\,\mathrm{ReLU}(W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}) + \boldsymbol{b}^{(2)}\Big) + \boldsymbol{b}^{(3)}\Big).$$

3.3 ReLU and its derivative

$\mathrm{ReLU}(u) = \max(0, u)$; its derivative equals 1 for $u > 0$ and 0 for $u < 0$. At $u = 0$ there is no ordinary derivative; implementations select a subgradient, usually 0.

3.4 Counting trainable parameters

Layer	Weights	Biases	Total
$784 \to 128$	100,352	128	100,480
$128 \to 64$	8,192	64	8,256
$64 \to 10$	640	10	650
Total			**109,386**

3.5 What neurons actually calculate

Neuron i in the first layer computes $z_i^{(1)} = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$, then $a_i^{(1)} = \max(0, z_i^{(1)})$. The weights can eventually respond to evidence at specific positions, while deeper combinations express more complicated digit patterns. These are learned responses, not human-coded stroke rules.

Chapter 4

Calculus and Backpropagation

4.1 Gradient descent

A derivative measures local sensitivity. Gradient descent uses $\theta \leftarrow \theta - \eta \nabla_\theta J$, where $\eta > 0$ is the learning rate. This is a local step; it is not a guarantee of finding the global minimum in a nonconvex neural network.

Worked Example

For $w = 0.5$, learning rate $\eta = 0.1$, and gradient $\partial L/\partial w = 0.8$, the update is $w_{\text{new}} = 0.5 - 0.1(0.8) = 0.42$.

4.2 Deriving the softmax Jacobian

Write $S = \sum_r e^{z_r}$. By quotient rule,

$$\frac{\partial p_i}{\partial z_j} = \frac{\delta_{ij} e^{z_i} S - e^{z_i} e^{z_j}}{S^2} \tag{4.1}$$

$$= p_i(\delta_{ij} - p_j), \tag{4.2}$$

where $\delta_{ij} = 1$ for $i = j$ and 0 otherwise. Hence the diagonal entries are $p_i(1 - p_i)$ and off-diagonal entries are $-p_i p_j$.

4.3 Why softmax plus cross-entropy simplifies

Since $L = -\sum_i y_i \log p_i$, $\partial L/\partial p_i = -y_i/p_i$. Chain rule yields

$$\frac{\partial L}{\partial z_j} = \sum_i \frac{\partial L}{\partial p_i} \frac{\partial p_i}{\partial z_j} = -\sum_i y_i(\delta_{ij} - p_j) \tag{4.3}$$

$$= -y_j + p_j \sum_i y_i = p_j - y_j. \tag{4.4}$$

Key Result

For one example, the output-layer error is exactly $\boxed{\boldsymbol{\delta}^{(3)} = \boldsymbol{p} - \boldsymbol{y}}$. This is the starting point for backpropagation.

4.4 Output-layer gradients

For $z_i^{(3)} = \sum_j W_{ij}^{(3)} a_j^{(2)} + b_i^{(3)}$, $\partial z_i^{(3)} / \partial W_{ij}^{(3)} = a_j^{(2)}$. Therefore,

$$\boxed{\nabla_{W^{(3)}} L = \boldsymbol{\delta}^{(3)} (\boldsymbol{a}^{(2)})^T, \quad \nabla_{\boldsymbol{b}^{(3)}} L = \boldsymbol{\delta}^{(3)}.} \tag{4.5}$$

The outer product has dimensions $(10 \times 1)(1 \times 64) = 10 \times 64$, exactly the weight-matrix shape.

4.5 Hidden-layer gradients

The chain rule and ReLU derivative produce

$$\boldsymbol{\delta}^{(2)} = ((W^{(3)})^T \boldsymbol{\delta}^{(3)}) \odot \mathrm{ReLU}'(\boldsymbol{z}^{(2)}), \tag{4.6}$$

$$\boldsymbol{\delta}^{(1)} = ((W^{(2)})^T \boldsymbol{\delta}^{(2)}) \odot \mathrm{ReLU}'(\boldsymbol{z}^{(1)}). \tag{4.7}$$

Then

$$\nabla_{W^{(2)}} L = \boldsymbol{\delta}^{(2)} (\boldsymbol{a}^{(1)})^T, \qquad \nabla_{\boldsymbol{b}^{(2)}} L = \boldsymbol{\delta}^{(2)}, \tag{4.8}$$

$$\nabla_{W^{(1)}} L = \boldsymbol{\delta}^{(1)} \boldsymbol{x}^T, \qquad \nabla_{\boldsymbol{b}^{(1)}} L = \boldsymbol{\delta}^{(1)}. \tag{4.9}$$

For every layer, update $W^{(\ell)} \leftarrow W^{(\ell)} - \eta \nabla_{W^{(\ell)}} L$ and similarly for the biases.

4.6 Small numerical backpropagation example

Take input $\boldsymbol{x} = (0.6, 0.8)^T$, two-class weights $W = \begin{bmatrix} 0.2 & -0.3 \\ 0.4 & 0.1 \end{bmatrix}$, and $\boldsymbol{b} = (0.1, -0.2)^T$. Then

$$W\boldsymbol{x} + \boldsymbol{b} = \begin{bmatrix} 0.12 - 0.24 + 0.1 \\ 0.24 + 0.08 - 0.2 \end{bmatrix} = \begin{bmatrix} -0.02 \\ 0.12 \end{bmatrix}. \tag{4.10}$$

Softmax gives $\boldsymbol{p} \approx (0.4651, 0.5349)^T$. Suppose true target $\boldsymbol{y} = (0, 1)^T$. Loss $L = -\log(0.5349) \approx 0.6256$ and

$$\boldsymbol{\delta} = \boldsymbol{p} - \boldsymbol{y} \approx \begin{bmatrix} 0.4651 \\ -0.4651 \end{bmatrix}. \tag{4.11}$$

Thus

$$\nabla_W L = \boldsymbol{\delta} \boldsymbol{x}^T \approx \begin{bmatrix} 0.2791 & 0.3721 \\ -0.2791 & -0.3721 \end{bmatrix}. \tag{4.12}$$

With $\eta = 0.1$, the updated weights are $W_{\text{new}} \approx \begin{bmatrix} 0.1721 & -0.3372 \\ 0.4279 & 0.1372 \end{bmatrix}$, and biases become $\boldsymbol{b}_{\text{new}} \approx (0.05349, -0.15349)^T$. With these *both* updated, the true-class probability increases to approximately 0.5684, so loss decreases to about 0.5649.

4.7 Gradient checking with finite differences

To debug an analytical gradient at component θ_j, compare to

$$g_j^{\text{numeric}} \approx \frac{L(\theta + \epsilon \boldsymbol{e}_j) - L(\theta - \epsilon \boldsymbol{e}_j)}{2\epsilon}. \tag{4.13}$$

For a suitable small ϵ (for example 10^{-5} in double precision), the numeric and backprop derivatives should agree closely away from nondifferentiable points.

Chapter 5

Convolutional Neural Networks

5.1 Local patterns and weight sharing

An MLP learns absolute-position features. A CNN uses a small learned kernel repeatedly across spatial positions; this gives translation equivariance to the convolution operation (subject to boundary effects), and makes it economical to detect similar features in different parts of an image.

5.2 A convolution-like operation: cross-correlation

A neural-network library usually evaluates cross-correlation, conventionally called convolution:

$$Y_{i,j} = \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} K_{u,v} X_{i+u,j+v} + b. \tag{5.1}$$

Pure mathematical convolution reverses the kernel in both directions. Since kernels are learned, the naming difference usually makes no difference to expressive power.

Worked Example

Let $X_{\text{patch}} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$ and $K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$. The dot product is $(1-1)+(0-1)+(1-0) = 0$. That is one output location. Sliding the kernel gives a full feature map.

5.3 Output-size mathematics

For an input size H, kernel K, padding P, stride S, and dilation D,

$$\boxed{H_{\text{out}} = \left\lfloor \frac{H + 2P - D(K-1) - 1}{S} + 1 \right\rfloor.} \tag{5.2}$$

The same formula applies to width. With $H = 28, K = 3, D = 1, S = 1, P = 0$, the result is 26. With $P = 1$, the result is 28.

5.4 Multiple input channels and output maps

For output channel o and input channel c,

$$Y_{o,i,j} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u,v} K_{o,c,u,v} X_{c,i+u,j+v}, \tag{5.3}$$

with padding and stride handled by adjusted indices. Number of trainable parameters:

$$\boxed{C_{\text{out}}(C_{\text{in}} K_h K_w + 1).} \tag{5.4}$$

A grayscale input with 32 filters of 3×3 requires $32(9+1) = 320$ parameters.

5.5 Activation and pooling

ReLU applies componentwise: $A_{o,i,j} = \max(0, Y_{o,i,j})$. For 2×2 max pooling with stride 2, each nonoverlapping block becomes its maximum.

Worked Example

For $A = \begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 0 & 2 & 8 & 3 \\ 1 & 4 & 2 & 7 \end{bmatrix}$, the pooled image is $P = \begin{bmatrix} 6 & 4 \\ 4 & 8 \end{bmatrix}$. The dimensions change from 4×4 to 2×2.

5.6 Full MNIST CNN: dimensions and parameter count

Choose padding one and stride one for both 3×3 convolutions, and 2×2 pooling after each.

Layer	Output shape	Parameters
Input	$1 \times 28 \times 28$	0
Conv2D $1 \to 32$	$32 \times 28 \times 28$	320
ReLU	$32 \times 28 \times 28$	0
MaxPool 2×2	$32 \times 14 \times 14$	0
Conv2D $32 \to 64$	$64 \times 14 \times 14$	18,496
ReLU	$64 \times 14 \times 14$	0
MaxPool 2×2	$64 \times 7 \times 7$	0
Flatten	3136	0
Dense $3136 \to 128$	128	401,536
ReLU	128	0
Dense $128 \to 10$	10	1,290
Total		**421,642**

The second convolution has $64(32 \cdot 9 + 1) = 18{,}496$ parameters. The dense layer has $3136 \cdot 128 + 128 = 401{,}536$. This particular CNN is parameter-heavy because of its dense classifier; other architectures can use global average pooling.

5.7 Backpropagation through convolution

For a single-channel, stride-one, no-padding cross-correlation $Y_{ij} = \sum_{uv} K_{uv} X_{i+u,j+v} + b$, define $G_{ij} = \partial L / \partial Y_{ij}$. Then

$$\frac{\partial L}{\partial K_{uv}} = \sum_{ij} G_{ij} X_{i+u,j+v}, \tag{5.5}$$

$$\frac{\partial L}{\partial b} = \sum_{ij} G_{ij}, \tag{5.6}$$

$$\frac{\partial L}{\partial X_{mn}} = \sum_{\substack{i,j,u,v \\ i+u=m,\ j+v=n}} G_{ij} K_{uv}. \tag{5.7}$$

For multiple channels, sum over output or input channels as appropriate. Crucially, the gradient for one shared kernel element accumulates contributions from all spatial locations.

5.8 Gradient through max pooling

For $y = \max(x_1, \ldots, x_4)$ with a unique maximum at position r, $\partial y / \partial x_r = 1$ and the other derivatives are zero. Therefore the upstream gradient routes to the winning input. Ties require a convention/-subgradient.

Chapter 6

Training Algorithms and Generalization

6.1 Mini-batch optimization

For batch size B, $L_B = \frac{1}{B}\sum_{i=1}^{B} L_i$, and $\nabla_\theta L_B = \frac{1}{B}\sum_i \nabla_\theta L_i$. A batch of 64 images is a tensor of shape $(64, 1, 28, 28)$, producing logits $(64, 10)$. An epoch over 60,000 images yields $\lceil 60000/64 \rceil = 938$ batches when the last incomplete batch is retained.

6.2 Batch matrix gradient formulas

Store MLP examples as columns, $X \in \mathbb{R}^{784\times B}$, with $Y, P \in \mathbb{R}^{10\times B}$. Use $\mathbf{1}_B$ for the vector of ones. Then $Z^{(1)} = W^{(1)}X + \boldsymbol{b}^{(1)}\mathbf{1}_B^T$. For *mean* cross-entropy, $\Delta^{(3)} = (P - Y)/B$ and

$$\nabla_{W^{(3)}} L_B = \Delta^{(3)}(A^{(2)})^T, \qquad \nabla_{\boldsymbol{b}^{(3)}} L_B = \Delta^{(3)}\mathbf{1}_B. \tag{6.1}$$

This describes efficient vectorized mini-batch training.

6.3 Adam optimizer

For gradient g_t and elementwise operations,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t, \tag{6.2}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2, \tag{6.3}$$

$$\hat{m}_t = m_t/(1 - \beta_1^t), \quad \hat{v}_t = v_t/(1 - \beta_2^t), \tag{6.4}$$

$$\theta_t = \theta_{t-1} - \eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \tag{6.5}$$

Common starting values are $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$. Actual choices must be validated empirically.

6.4 Initialization and regularization

He normal initialization uses $w \sim \mathcal{N}(0, 2/n_{\text{in}})$ for ReLU layers. For $n_{\text{in}} = 784$, the standard deviation is $\sqrt{2/784} \approx 0.0505$. A typical L2-penalized objective is

$$J_{\text{reg}} = J_{\text{CE}} + \frac{\lambda}{2}\sum_\ell \|W^{(\ell)}\|_F^2, \quad \nabla_W J_{\text{reg}} = \nabla_W J_{\text{CE}} + \lambda W. \tag{6.6}$$

Decoupled weight decay (e.g., AdamW) is not exactly identical to this penalty under adaptive optimization. Inverted dropout during training uses $\tilde{a}_i = m_i a_i / q$ with $m_i \sim \text{Bernoulli}(q)$, and is switched off for evaluation. Small label-preserving rotations, shifts, and deformations provide useful augmentation; extreme transforms can change a digit's meaning.

6.5 Train, validation and test sets

Use training examples to fit weights, validation examples to choose settings and stop training, and the held-out test set only for final evaluation. Repeatedly choosing configurations based on test accuracy leaks test information into the workflow.

Chapter 7

Evaluation, Inference and Implementations

7.1 Accuracy and confusion matrix

If 9,850 of 10,000 test predictions are correct, accuracy is 98.5%; this is a hypothetical example, not an empirical result for this book's model. Define C_{ij} as number of true-digit-i examples predicted as j. Then

$$\text{Accuracy} = \frac{\sum_i C_{ii}}{\sum_{ij} C_{ij}}. \tag{7.1}$$

For a class k, Precision $= TP/(TP+FP)$, Recall $= TP/(TP+FN)$ and $F_1 = 2PR/(P+R)$ (when denominators are nonzero).

7.2 New-image inference pipeline

Read an image, convert to grayscale, match its foreground/background convention, align the digit sensibly, resize/preprocess consistently with training, normalize to $[0,1]$, and create a tensor of shape $(1,1,28,28)$. The trained model outputs logits $\boldsymbol{z}$; probabilities are $\text{softmax}(\boldsymbol{z})$ and prediction is $\arg\max_k z_k$. No gradient update occurs at inference time.

7.3 Clear PyTorch implementation

The following illustrates the architecture and training equations. It requires `torch` and `torchvision`; downloading MNIST requires a working dataset source/network.

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

train_data = datasets.MNIST(
    root="./data", train=True, download=True,
    transform=transforms.ToTensor())
test_data = datasets.MNIST(
    root="./data", train=False, download=True,
    transform=transforms.ToTensor())
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=256)
```

```
class DigitCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(), nn.MaxPool2d(2),
            nn.Flatten(), nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(), nn.Linear(128, 10))
    def forward(self, x):
        return self.net(x)  # raw logits, no softmax

model = DigitCNN()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
for epoch in range(3):
    model.train()
    for images, labels in train_loader:
        optimizer.zero_grad()
        logits = model(images)
        loss = criterion(logits, labels)
        loss.backward()
        optimizer.step()
    print("Epoch", epoch + 1, "last batch loss", loss.item())

model.eval()
correct, total = 0, 0
with torch.no_grad():
    for images, labels in test_loader:
        pred = model(images).argmax(dim=1)
        correct += (pred == labels).sum().item()
        total += labels.numel()
print("Test accuracy:", correct / total)
```

Important Note

A reported test result requires actually executing the code, saving the random seed, recording software versions, and preserving the evaluation protocol. These notes do not assert an experimentally measured accuracy.

Chapter 8

Review Exercises with Fully Worked Answers

Exercise 1: input dimension

A normalized MNIST image is flattened for a dense layer. How long is the vector? **Answer:** $28 \cdot 28 = 784$. A batch of B column vectors becomes $784 \times B$.

Exercise 2: linear classifier parameters

How many trainable parameters in $784 \to 10$? **Answer:** $784 \cdot 10 + 10 = 7{,}850$.

Exercise 3: cross-entropy

True class probability is 0.2. Compute loss and interpret. **Answer:** $L = -\log(0.2) = \log(5) \approx 1.6094$. Lower correct-class probability means higher loss.

Exercise 4: output layer gradient

For $\boldsymbol{p} = (0.1, 0.6, 0.3)^T$, target $\boldsymbol{y} = (0, 1, 0)^T$, compute gradient with respect to logits. **Answer:** $\boldsymbol{\delta} = \boldsymbol{p} - \boldsymbol{y} = (0.1, -0.4, 0.3)^T$.

Exercise 5: filter output size

Find spatial output for 28×28, $K = 5, P = 0, S = 1, D = 1$. **Answer:** $(28 - 5)/1 + 1 = 24$, so 24×24 per filter.

Exercise 6: convolution parameter count

A layer has $C_{\text{in}} = 16$, $C_{\text{out}} = 32$, and a 3×3 kernel. **Answer:** $32(16 \cdot 9 + 1) = 4{,}640$.

Exercise 7: mini-batch count

How many batches for 60,000 samples and batch size 128, retaining the smaller final batch? **Answer:** $\lceil 60000/128 \rceil = 469$.

Exercise 8: pooling

Pool $\begin{bmatrix} 2 & 5 \\ 7 & 4 \end{bmatrix}$ by 2×2 max pooling. **Answer:** output 7, and in backward pass the incoming gradient is assigned to the lower-left input only, assuming a unique maximum.

Exercise 9: why divide the gradient by B?

The batch objective is an *average*. Taking its derivative gives $\nabla L_B = \frac{1}{B} \sum_i \nabla L_i$. If we omit $1/B$, changing batch size changes the gradient magnitude, unless the loss is intentionally defined as a sum.

Exercise 10: finite-difference check

For scalar $L(w) = (w-3)^2$, find derivative at $w = 1$. **Answer:** $dL/dw = 2(w-3) = -4$. A small step in the negative gradient direction increases w, moving toward 3.

Chapter 9

Final Formula Sheet

$$
\begin{aligned}
\text{Normalize:}\quad & X' = X/255 \\
\text{Dense:}\quad & \boldsymbol{z} = W\boldsymbol{x} + \boldsymbol{b} \\
\text{ReLU:}\quad & \mathrm{ReLU}(z) = \max(0, z) \\
\text{Softmax:}\quad & p_k = e^{z_k} / \sum_j e^{z_j} \\
\text{Cross-entropy:}\quad & L = -\sum_k y_k \log p_k \\
\text{Output gradient:}\quad & \frac{\partial L}{\partial \boldsymbol{z}} = \boldsymbol{p} - \boldsymbol{y} \\
\text{Dense weight gradient:}\quad & \nabla_W L = \boldsymbol{\delta}\boldsymbol{a}^T \\
\text{Hidden error:}\quad & \boldsymbol{\delta}^{(\ell)} = ((W^{(\ell+1)})^T \boldsymbol{\delta}^{(\ell+1)}) \odot f'(\boldsymbol{z}^{(\ell)}) \\
\text{Gradient step:}\quad & \theta \leftarrow \theta - \eta \nabla_\theta L \\
\text{Conv parameters:}\quad & C_{\text{out}}(C_{\text{in}} K_h K_w + 1) \\
\text{Prediction:}\quad & \hat{y} = \arg\max_k p_k.
\end{aligned}
$$

Closing perspective

A learned handwritten-digit classifier converts grayscale pixels into features and class scores. Its loss measures disagreement with known labels; backpropagation calculates how every parameter contributes to that disagreement; optimization modifies those parameters. The same core ideas underpin contemporary image classification and visual-perception systems.